The Hut Participation Portal

Supporting Documentation

Design · Testing · References · Report

Repository: Abhinavkk99/hut-attendance-v2

Generated from the project documentation suite

Project Report

The Hut Participation Portal — Project Report

> A consolidated report covering methodology, research, design, implementation, security analysis, and testing. Detailed material lives in the companion documents referenced throughout: DESIGN.md, TESTING.md, REFERENCES.md, and the repository `CHANGELOG.md`.

---

1. Introduction

The Hut Participation Portal is a web application that helps a South Australian community organisation register participants, enrol them in programs, record attendance, and report on participation. It targets three kinds of users — **Staff**, **Managers**, and **Administrators** — each with a different level of access.

This report documents the work undertaken on the portal: aligning it with an updated reference design, implementing a participation-lifecycle feature set, and performing a security review of the user-approval system with subsequent hardening.

1.1 Objectives

1. Bring the application in line with an updated reference version that introduced a participation-history capability.
2. Implement that capability (enrol / withdraw / deactivate / reactivate) without losing data the organisation needs for reporting.
3. Review the security of the account-approval flow and remediate any weaknesses.
4. Produce supporting documentation to professional standards.

---

2. Methodology

The work followed a structured, evidence-based approach rather than ad-hoc changes.

2.1 Comparative analysis (alignment task)

A newer reference build of the application was provided. To decide what to change, a **file-by-file comparison** was performed between the current codebase and the reference:

1. **Inventory** — listed and diffed the file trees to find added/removed files.
2. **Change sizing** — measured line-count deltas per file to locate the substantive differences (e.g. the reference's much larger Training, Reports, and profile pages).
3. **Functional extraction** — for each significantly changed file, the *functional* differences (data fields, database queries, business logic) were separated from cosmetic ones (styling, comments), and the **direction** of each change was noted (present in reference vs present in current).
4. **Conflict identification** — places where the current app was *ahead* of the reference were catalogued so they would not be regressed.
5. **Prioritisation** — changes were grouped into themes; the *participation-lifecycle foundation* was identified as the backbone other features depend on and was implemented first.

This produced a clear, justified work list before any code was written.

2.2 Security review

The approval feature was reviewed using a **threat-modelling** mindset — asking "how could an attacker abuse this?" — across the relevant code and database policies:

- the sign-up path (`SignUp.tsx`) and the database trigger that creates profiles;
- the authentication/approval gate (`AuthContext.tsx`);
- the row-level-security (RLS) policies on every table (`supabase-setup.sql`);
- the approval UI (`Approvals.tsx`) and its route protection.

Findings were classified by severity and mapped to standard categories (broken access control, privilege escalation). Reference: OWASP Top 10 access-control guidance.

2.3 Implementation & verification

Changes were made incrementally and verified by:

- a production build (`npm run build`),
- a TypeScript type-check pass (`tsc --noEmit` , since Vite does not type-check),
- and the manual test cases in TESTING.md.

All work was tracked in Git with descriptive commit messages (see `CHANGELOG.md`).

---

3. Background & Research

- **Architecture pattern:** the app is a single-page application backed by Supabase (PostgreSQL + Auth + RLS), a common "backend-as-a-service" pattern that removes the need for a bespoke server while still enforcing security at the database layer.
- **Authorisation model:** Role-Based Access Control (RBAC) — a widely used industry pattern — implemented through three roles and enforced by database policies.
- **Row-Level Security:** PostgreSQL RLS allows per-row authorisation rules. Research into Supabase's RLS guidance highlighted a known pitfall — policies that query the same table they protect can recurse infinitely — which directly informed the use of `SECURITY DEFINER` helper functions in the fix (see §6).
- **Soft-delete / audit history:** retaining records with status flags rather than deleting them is standard practice where historical reporting and auditability are required, which is the case for community-program participation data.

See REFERENCES.md for the full list of libraries and sources.

---

4. Design Summary

The system is a client-side React application talking directly to Supabase. Its data model centres on **participants**, **programs**, and the **enrolments** linking them, with **attendance records**, **staff assignments**, and a **participation history** log. Access is governed by the three-tier role model.

The full design — architecture diagram, entity-relationship diagram, role matrix, workflow/sequence diagrams, key decisions and assumptions — is documented in DESIGN.md. Key design decisions are summarised there in §7; the most significant are:

- RLS (not the UI) is the authoritative security boundary.
- A soft-delete lifecycle preserves history.
- Roles are assigned server-side, never from user input.
- Dates are normalised to noon UTC to avoid timezone day-shift bugs.

---

5. Implementation Summary

The participation-lifecycle foundation was implemented across the database and four pages:

Area	Change
Database	New migration adding lifecycle columns, a `participation_history` table, and triggers that auto-log changes.
Participant profile	Hard-delete replaced with soft **withdraw / deactivate / reactivate / re-enrol**, each with a date and confirmation; a "Past Programs" section.
Add to Program	Enrolment-date picker; stores `start_date` and active status.
Attendance	Lists programs for the chosen date (bug fix); shows only active enrolees.
Participant search	Active/Inactive classification based on active enrolments.

Conflicting improvements already present in the current app were deliberately **preserved** (e.g. an additional participant field, schema-resilient database writes, a safeguard that detects permission-blocked approvals). Full details are in `CHANGELOG.md`.

---

6. Security Analysis & Hardening

The review of the approval feature found four issues, all since fixed (`security-fixes.sql` + route guards). Summarised:

#	Issue	Severity	Fix
1	Sign-up trusted a client-supplied role, allowing self-assigned **admin**	High	Trigger hardcodes `role='staff'`; roles elevated only in the database
2	Participant data tables were world-accessible (`USING (true)`), so the public anon key alone could read/write personal data	Critical	Approval- and role-scoped RLS policies on every table
3	Any logged-in user could view all profiles and the approval list	Medium	Profile visibility restricted to owner or admin
4	Denying a sign-up left an orphaned authentication account	Low	Trigger removes the `auth.users` row on profile deletion

A notable implementation detail: the role/approval checks were placed in `SECURITY DEFINER` helper functions (`is_approved()`, `has_role()`) so that policies on the `profiles` table do not recursively invoke themselves — avoiding the infinite-recursion pitfall identified during research (§3).

A non-technical summary of these issues, suitable for stakeholders, is available on request.

---

7. Testing

Testing is **manual**, organised as a structured test plan covering authentication and approval, role-based access (both UI guards and database policies), participant registration, the enrolment lifecycle, search classification, and attendance. Each case lists pre-conditions, steps, and the expected result for the tester to record.

The plan also captures **debugging notes** for the seven defects found and fixed during development (including the attendance date bug, the timezone day-shift, and the two security defects). Static verification — a clean production build and a TypeScript type-check pass — was performed and recorded.

Full detail: TESTING.md.

---

8. Conclusion & Future Work

The portal now has a robust participation-lifecycle capability with a full audit trail, and the account-approval flow has been hardened so that authorisation is enforced at the database level rather than only in the interface.

Deferred / future work (scoped but not yet implemented):

- The fitness/health-information capture workflow.
- The expanded demographic reporting and spreadsheet export.
- An interactive training walkthrough.
- Automated (unit/integration) test coverage to complement the manual plan.

Operational note: the security fixes and lifecycle features require the SQL scripts to be applied to the live database in order (see DESIGN.md §9); the application code changes are already deployed via Git.

---

9. References

See REFERENCES.md for the complete list of libraries (with versions and licences), design-system sources, tools, and documentation consulted.

Design Document

Design Document — The Hut Participation Portal

1. Overview

The Hut Participation Portal is a web application for a South Australian community organisation to manage program participants, enrol them in programs, and record attendance. It replaces paper/spreadsheet processes with a role-based system that keeps participant records, enrolment history, and attendance in one place.

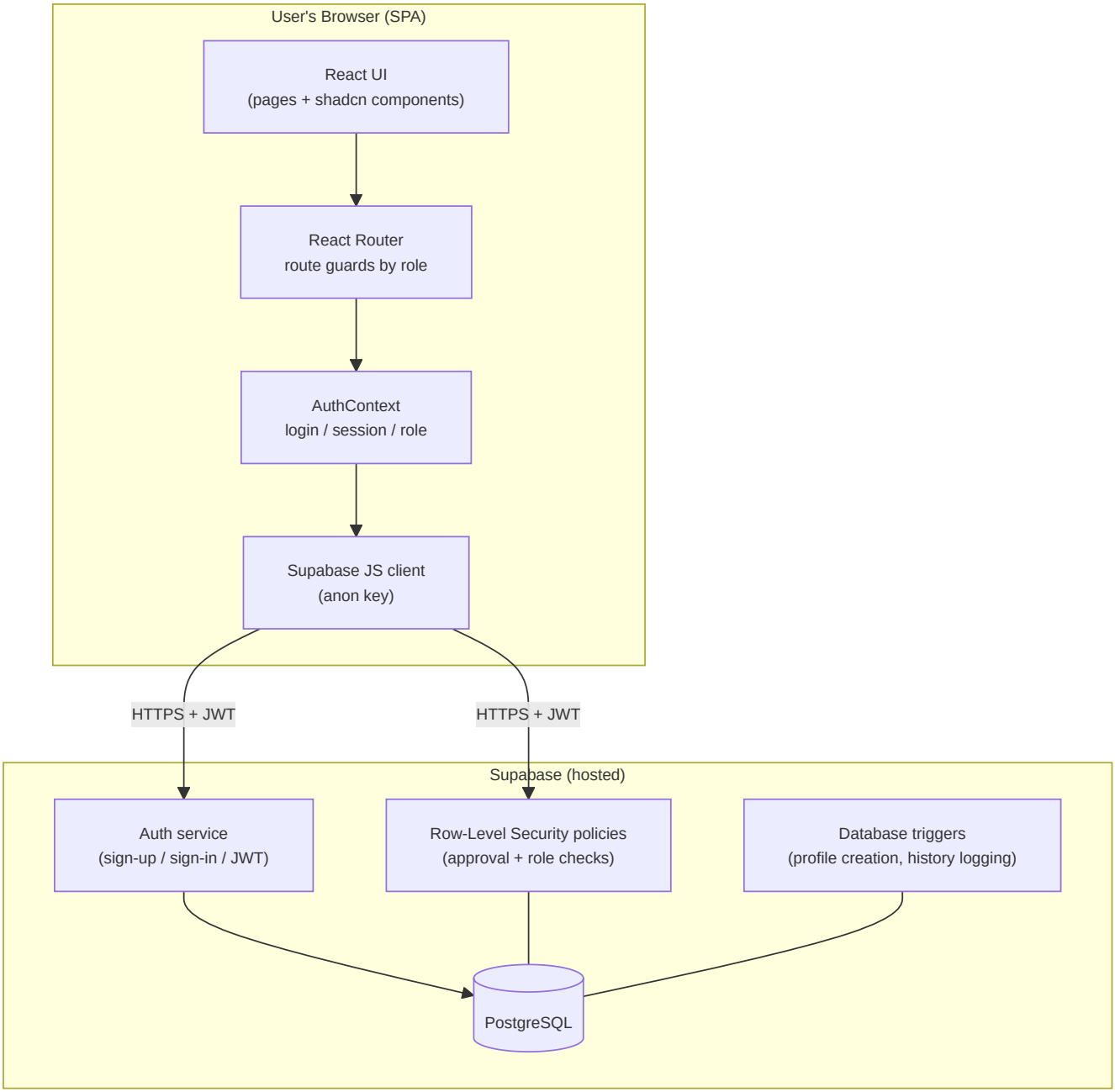
The application is a **single-page application (SPA)**: there is no custom backend server. The browser talks directly to **Supabase** (a hosted PostgreSQL database with built-in authentication and row-level security), which is the system's data store, authentication provider, and security boundary.

2. Technology Stack

Layer	Technology	Purpose
UI framework	React 18 + TypeScript	Component-based interface, type safety
Build tool	Vite 6	Dev server + production bundling
Routing	React Router 7 (data router)	Client-side navigation & route guards
Styling	Tailwind CSS v4	Utility-first styling
UI components	shadcn/ui (Radix UI primitives)	Accessible dialogs, selects, etc.
Backend-as-a-service	Supabase (PostgreSQL, Auth, RLS)	Database, authentication, authorisation
Charts	Recharts	Reporting visualisations

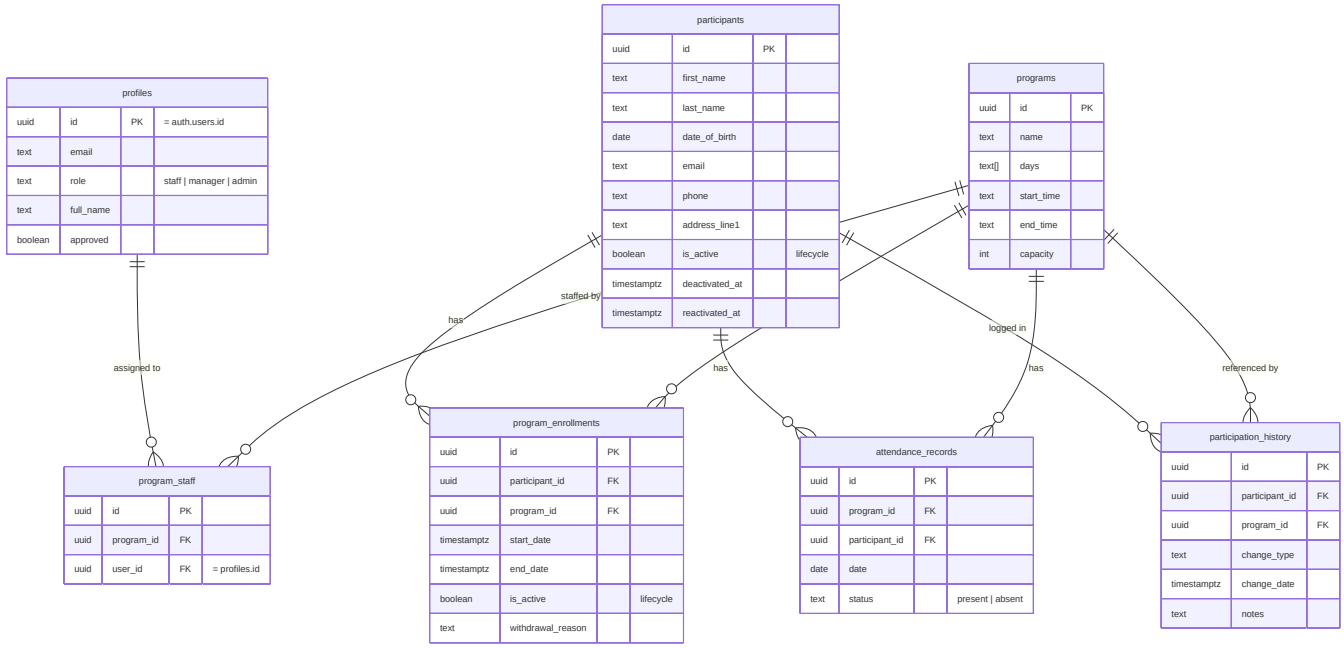
Full version and licence details are in REFERENCES.md.

3. System Architecture



Key point: because the browser holds only the public *anon key*, the database's Row-Level Security (RLS) policies — not the UI — are what actually enforce who can read or change data. The UI's role checks are a usability layer on top.

4. Data Model



A `UNIQUE(participant_id, program_id)` constraint on `program_enrollments` means a participant has at most one enrolment row per program; re-joining a program reuses that row rather than creating a duplicate (see §6.3).

5. Roles & Access Control

The system has a **3-tier role model**. Access widens from Staff → Manager → Admin.

Capability	Staff	Manager	Admin
Mark attendance (assigned programs only for staff)	✅	✅	✅
View training	✅	✅	✅
Register participants / enrol in programs	❌	✅	✅
Search participants / view profiles	❌	✅*	✅
Manage programs & assign staff	❌	❌	✅
View reports	❌	❌	✅
Approve / deny new user sign-ups	❌	❌	✅

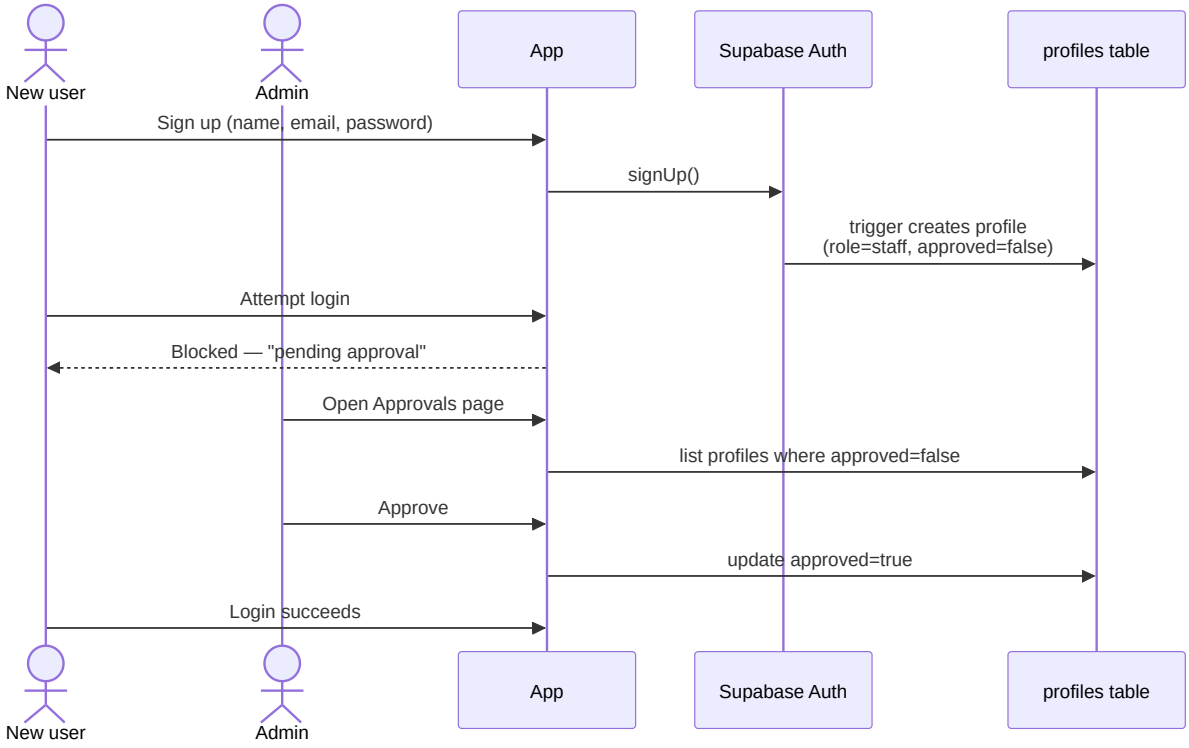
* Managers can open participant profiles they enrol into; full participant search is admin-only.

Access is enforced in **two layers**:

1. **Database (authoritative):** RLS policies check that the caller is an *approved* user with the required role before any read/write (see security-fixes.sql).
2. **Application (usability):** the sidebar hides links and route guards redirect unauthorised users (`ProtectedRoute roles={...}` in `routes.tsx`).

6. Core Workflows

6.1 Sign-up and approval



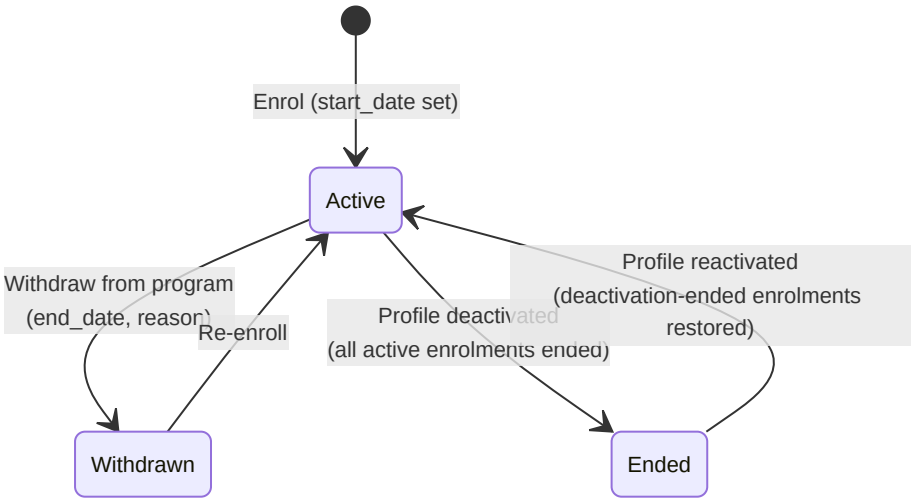
New accounts are always created as **unapproved Staff**. An administrator reviews and approves them; only then can they log in. Higher roles are assigned manually in the database, never self-selected (a security decision — see §7).

6.2 Register a participant and enrol in a program

A Manager/Admin registers a participant (multi-step form capturing personal, cultural, emergency-contact and program-specific details), then enrols them in a program via *Add to Program*, choosing an enrolment date.

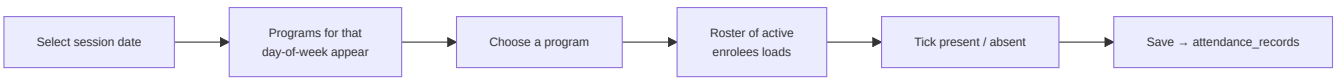
6.3 Participation lifecycle (enrol / withdraw / deactivate / reactivate)

Rather than deleting records (which would lose history), the system uses a **soft-delete lifecycle**. Enrolments and participant profiles carry an `is_active` flag; changes are timestamped and logged to `participation_history` automatically by database triggers.



This preserves a full audit trail: who was enrolled when, why they left, and when they returned.

6.4 Mark attendance



Staff see only the programs they are assigned to (`program_staff`); managers and admins see all programs.

7. Significant Decisions

Decision	Rationale
No custom backend; client + Supabase RLS	Faster to build; RLS gives database-level security without a server to maintain.
RLS is the security boundary, not the UI	The anon key is public (it ships in the browser bundle); data is safe only because RLS restricts every query.
Soft-delete lifecycle instead of hard delete	Community programs need historical reporting (who attended, withdrawals, re-enrolments); deleting rows would destroy that.
Role assigned server-side, never from sign-up input	Prevents a user from registering themselves as an admin via the public API.
Approval gate for new accounts	Staff handle vulnerable participants' data; accounts must be vetted before access.
Dates normalised to noon UTC before storing	A bare `YYYY-MM-DD` can shift by a day across timezones; noon UTC avoids off-by-one errors.
Schema-resilient insert/update (retry stripping unknown columns)	The schema evolved over time; this lets the app run against databases that haven't had every migration applied yet.

8. Assumptions

- The organisation operates in South Australia (reference data: SA council regions, Adelaide Hills townships).
- Supabase email confirmation is **disabled**, so the custom approval gate governs access instead.
- The **first administrator** is approved/elevated manually in the Supabase dashboard.
- A single Supabase project backs the app; credentials live in `src/config/supabase.config.ts` .
- The published anon key is safe to expose **provided** the RLS policies in `security-fixes.sql` are applied.

9. Database Setup Order

Scripts must be run in the Supabase SQL Editor in this order (all idempotent):

1. `supabase-setup.sql` — tables, base policies, sign-up trigger.
2. `historical-participation-tracking.sql` — lifecycle columns, `participation_history` , logging triggers.
3. `security-fixes.sql` — hardened RLS and sign-up trigger (see REPORT.md §Security).

10. Repository Map (high level)

```
src/
  main.tsx                App bootstrap
  app/
    App.tsx               RouterProvider
    routes.tsx            Route table + role guards
    context/AuthContext   Auth state & role
    components/Layout     Sidebar/topbar + nav by role
    components/ui/        shadcn/ui primitives
    pages/                One file per screen
    utils/constants.ts    SA reference data
  lib/supabase.ts         Supabase client + DB types
  config/supabase.config  Project URL + anon key
*.sql                     Database setup & migrations
docs/                     This documentation
```

Test Plan & Results

Test Plan & Results — The Hut Participation Portal

1. Purpose & Scope

This document defines manual test cases for the portal, covering authentication, role-based access, the participation lifecycle, attendance, and the security hardening. It also records debugging notes for defects found and fixed during development.

> **How to read the Result column:** "Expected" describes the correct behaviour. Record the **Actual** outcome and mark **Pass/Fail** when you execute each case. Cases are written to be executed manually in a browser against a Supabase instance with all three SQL scripts applied (see DESIGN.md §9).

2. Test Environment

Item	Value
Build command	`npm run build`
Dev server	`npm run dev`
Browser	Chrome / Firefox (latest)
Backend	Supabase project with `supabase-setup.sql`, `historical-participation-tracking.sql`, `security-fixes.sql` applied
Test accounts	One each of: unapproved user, Staff, Manager, Admin

3. Test Data Setup

1. Create an Admin: sign up, then in Supabase set `role='admin'`, `approved=true`.
2. Create a Manager and a Staff account; approve both; set roles accordingly.
3. Assign the Staff account to at least one program (`program_staff`).
4. Register 2–3 participants and enrol them in programs.

4. Test Cases

4.1 Authentication & Approval

ID	Pre-condition	Steps	Expected Result	Actual	P/F
AUTH-01	New user	Sign up with name/email/password	Account created; message indicates approval pending		
AUTH-02	Unapproved account	Attempt login	Login blocked with "pending approval" message; not taken to a dashboard		
AUTH-03	Admin logged in	Open Approvals; approve the pending user	User disappears from pending list		
AUTH-04	Newly approved user	Log in	Login succeeds; lands on a dashboard		
AUTH-05	Admin	Deny a pending user	User removed from list; profile deleted; email freed for re-registration		
AUTH-06	Logged in	Click Sign out	Session ends; redirected to /login; back button does not restore the app		

4.2 Role-Based Access (UI guards)

ID	Role	Steps	Expected Result	Actual	P/F
RBAC-01	Staff	Inspect sidebar	Only Attendance + Training (plus dashboard) visible		
RBAC-02	Manager	Inspect sidebar	Staff items + Register + Add to Program		
RBAC-03	Admin	Inspect sidebar	All items incl. Participants, Programs, Reports, Approvals		
RBAC-04	Staff	Manually type `/reports` in the URL	Redirected away (to home), not shown the page		
RBAC-05	Manager	Manually type `/approvals` in the URL	Redirected away, not shown the page		
RBAC-06	Staff	Manually type `/programs` in the URL	Redirected away		

4.3 Role-Based Access (database — the real boundary)

ID	Role	Steps	Expected Result	Actual	P/F
RLS-01	Unauthenticated (anon key only)	Query `participants` directly via the Supabase REST endpoint	Request returns no rows / permission denied		
RLS-02	Staff	Attempt to insert a participant via the API	Rejected by RLS (managers/admins only)		
RLS-03	Manager	Insert a participant via the app	Succeeds		
RLS-04	Staff	Attempt to read another user's profile via the API	Returns only own profile		
RLS-05	Approved Staff	Read programs / mark attendance	Succeeds		

4.4 Participant Registration

ID	Pre-condition	Steps	Expected Result	Actual	P/F
REG-01	Manager	Complete the multi-step registration with all required fields	Participant saved; appears in search		
REG-02	Manager	Submit with a required field blank	Validation prevents submission; field flagged		
REG-03	Manager	Register against a DB missing an optional column	Insert still succeeds (schema-resilient retry strips unknown columns)		

4.5 Enrolment & Participation Lifecycle

ID	Pre-condition	Steps	Expected Result	Actual	P/F
LIFE-01	Manager, participant exists	Add to Program with an enrolment date	Enrolment created; `start_date` / `is_active=true` stored		
LIFE-02	Participant enrolled in all programs	Open Add to Program	Programs already enrolled are hidden; message shown		
LIFE-03	Participant with active enrolment	On profile, Withdraw from one program (pick date)	Program moves to "Past Programs"; attendance history retained		
LIFE-04	Participant with a past program	Re-enroll from "Past Programs"	Program returns to active; end date/reason cleared		
LIFE-05	Active participant	Deactivate profile (pick date)	Profile marked inactive; all active enrolments ended; banner shown		
LIFE-06	Inactive participant	Reactivate profile	Profile active again; programs ended *by deactivation* restored; individually-withdrawn ones stay inactive		
LIFE-07	After any lifecycle action	Check `participation_history` in DB	A matching row (enrollment/withdrawal/deactivation/reactivation) was logged automatically		

4.6 Search & Classification

ID	Pre-condition	Steps	Expected Result	Actual	P/F
SRCH-01	Admin	Open Participants; search by name/email/phone	Matching participants listed		
SRCH-02	Participant with ≥1 active enrolment	Check classification	Listed under **Active**		
SRCH-03	Participant whose enrolments are all withdrawn	Check classification	Listed under **Inactive**		

4.7 Attendance

ID	Pre-condition	Steps	Expected Result	Actual	P/F
ATT-01	Staff assigned to a program	Open Attendance	Only assigned programs appear		
ATT-02	Manager/Admin	Open Attendance	All programs appear		
ATT-03	Any	Change the session date to a different weekday	Program list updates to that day's programs (regression test for ATT bug — see §5)		
ATT-04	Program with active + withdrawn enrolees	Open roster	Only active enrolees shown		
ATT-05	Roster loaded	Mark some present, save	`attendance_records` created with correct date/status; success shown		

5. Debugging Notes (defects found & fixed)

#	Defect	Root cause	Fix	Verified by
D-1	Attendance always showed *today's* programs even when a different session date was picked	Program list was filtered once by `new Date().getDay()` and never recomputed on date change	Derived the program list reactively from the selected date's day-of-week; clear selection if the chosen program isn't on that day	ATT-03
D-2	Enrolment / withdrawal dates could store the wrong calendar day	A bare `YYYY-MM-DD` parsed as UTC midnight shifts a day in some timezones	Normalise all chosen dates to **noon UTC** before storing	LIFE-01, LIFE-03
D-3	Withdrawing a participant destroyed their history	Action used a hard `DELETE` on the enrolment row	Replaced with soft-delete (`is_active=false` + end date + reason); history preserved and logged	LIFE-03, LIFE-07
D-4	Inactive participants misclassified as Active in search	Classification counted *all* enrolments, including withdrawn ones	Count only `is_active=true` enrolments	SRCH-03
D-5	**Security:** a user could register themselves as admin via the public API	Sign-up trigger trusted a client-supplied `role` value	Trigger now hardcodes `role='staff'`; roles elevated only in the DB	RLS-02, AUTH-04
D-6	**Security:** participant data reachable with the public anon key	Data tables had open `USING (true)` policies	Replaced with approval/role-scoped RLS	RLS-01, RLS-02
D-7	Risk of RLS infinite recursion when policies query `profiles`	A policy on `profiles` selecting from `profiles`	Role/approval checks moved into `SECURITY DEFINER` helper functions that bypass RLS	RLS-04, RLS-05

6. Build & Static Verification (performed)

Check	Command	Result
Production build compiles	`npm run build`	✅ Pass — bundle produced, no errors
Type checking of changed files	`tsc --noEmit` (with project flags)	✅ Pass — no type errors in edited files

> Note: the project uses Vite/esbuild, which does not type-check during build, so a separate `tsc` pass was run to validate types.

> There is no automated unit-test suite in this project; the cases above are executed manually.

7. Known Limitations / Out of Scope

- No automated (unit/integration) tests are configured; testing is manual.
- Deactivating an `auth.users` account on "Deny" relies on a database trigger; in a hardened production setup this is often done via a service-role function instead.
- Reporting demographics depend on the optional participant fields being captured at registration.

References & Attributions

References & Attributions

This project builds on open-source libraries, design-system components, and external tools. All third-party software is used under its respective open-source licence, and all are attributed below. Versions reflect `package.json` / `package-lock.json`.

1. Core Framework & Build Tooling

Library	Version	Licence	Purpose
react	18.3.1	MIT	UI rendering library
react-dom	18.3.1	MIT	React DOM renderer
react-router	7.13.0	MIT	Client-side routing & route guards
typescript (via tooling)	—	Apache-2.0	Static typing
vite	6.3.5	MIT	Dev server & production bundler
@vitejs/plugin-react	4.7.0	MIT	React support for Vite
@tailwindcss/vite	4.1.12	MIT	Tailwind integration for Vite
tailwindcss	4.1.12	MIT	Utility-first CSS framework
tw-animate-css	1.3.8	MIT	Animation utilities for Tailwind

2. Backend / Data

Library	Version	Licence	Purpose
@supabase/supabase-js	^2.99.2	MIT	Client for Supabase (Postgres, Auth, RLS)

Supabase itself (the hosted PostgreSQL + Auth + Row-Level Security platform) is the project's backend-as-a-service. Documentation: https://supabase.com/docs

3. UI Component Primitives (Radix UI)

All Radix UI primitives are used under the **MIT** licence. They provide the accessible, unstyled building blocks wrapped by the shadcn/ui components in `src/app/components/ui/`.

Library	Version
@radix-ui/react-accordion	1.2.3
@radix-ui/react-alert-dialog	1.1.6
@radix-ui/react-aspect-ratio	1.1.2
@radix-ui/react-avatar	1.1.3
@radix-ui/react-checkbox	1.1.4
@radix-ui/react-collapsible	1.1.3
@radix-ui/react-context-menu	2.2.6
@radix-ui/react-dialog	1.1.6
@radix-ui/react-dropdown-menu	2.1.6
@radix-ui/react-hover-card	1.1.6
@radix-ui/react-label	2.1.2
@radix-ui/react-menubar	1.1.6
@radix-ui/react-navigation-menu	1.2.5
@radix-ui/react-popover	1.1.6
@radix-ui/react-progress	1.1.2
@radix-ui/react-radio-group	1.2.3
@radix-ui/react-scroll-area	1.2.3
@radix-ui/react-select	2.1.6
@radix-ui/react-separator	1.1.2
@radix-ui/react-slider	1.2.3
@radix-ui/react-slot	1.1.2
@radix-ui/react-switch	1.1.3
@radix-ui/react-tabs	1.1.3
@radix-ui/react-toggle	1.1.2
@radix-ui/react-toggle-group	1.1.2
@radix-ui/react-tooltip	1.1.8

4. UI Helpers, Styling & Interaction

Library	Version	Licence	Purpose
class-variance-authority	0.7.1	Apache-2.0	Variant-based component styling
clsx	2.1.1	MIT	Conditional className joining
tailwind-merge	3.2.0	MIT	Merge/de-duplicate Tailwind classes
lucide-react	0.487.0	ISC	Icon set
sonner	2.0.3	MIT	Toast notifications
cmdk	1.1.1	MIT	Command-menu component
vaul	1.1.2	MIT	Drawer component
next-themes	0.4.6	MIT	Theme (dark/light) management
input-otp	1.4.2	MIT	One-time-passcode input
react-day-picker	8.10.1	MIT	Date picker
react-hook-form	7.55.0	MIT	Form state & validation
react-resizable-panels	2.1.7	MIT	Resizable layout panels
@popperjs/core	2.11.8	MIT	Positioning engine
react-popper	2.3.0	MIT	React bindings for Popper
embla-carousel-react	8.6.0	MIT	Carousel
react-slick	0.31.0	MIT	Carousel/slider
react-responsive-masonry	2.7.1	MIT	Masonry grid layout
react-dnd	16.0.1	MIT	Drag-and-drop
react-dnd-html5-backend	16.0.1	MIT	HTML5 backend for react-dnd
canvas-confetti	1.9.4	ISC	Celebratory confetti effect
motion	12.23.24	MIT	Animation library
@emotion/react	11.14.0	MIT	CSS-in-JS (MUI dependency)
@emotion/styled	11.14.1	MIT	Styled API for Emotion
@mui/material	7.3.5	MIT	Material UI components
@mui/icons-material	7.3.5	MIT	Material UI icon set

5. Data Visualisation & Dates

Library	Version	Licence	Purpose
recharts	2.15.2	MIT	Charts for the Reports page
date-fns	3.6.0	MIT	Date formatting/manipulation

6. Design System & Assets

Source	Licence / Terms	Usage
shadcn/ui	MIT	Component patterns in `src/app/components/ui/` (built on Radix UI + Tailwind)
Unsplash	Unsplash Licence	Placeholder photography
Figma Make	Vendor tooling	Initial UI scaffold was exported from a Figma Make project (`figma:asset/` imports resolved in `vite.config.ts`)

(See also `ATTRIBUTIONS.md` in the repository root.)

7. Tools & Platforms

Tool	Purpose
Git / GitHub	Version control and hosting (`Abhinavkk99/hut-attendance-v2`)
Supabase	Database, authentication, row-level security
Vercel	Deployment (`hut-attendance-v2.vercel.app`)
npm	Dependency management

8. Documentation & Standards Consulted

- React documentation — https://react.dev
- React Router documentation — https://reactrouter.com
- Supabase Auth & Row-Level Security guides — https://supabase.com/docs/guides/auth
- PostgreSQL Row Security Policies — https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- OWASP guidance on access control & privilege escalation — https://owasp.org/Top10/
- Tailwind CSS documentation — https://tailwindcss.com/docs

Licence Summary

The third-party libraries are distributed under permissive open-source licences — predominantly **MIT**, with **ISC** (lucide-react, canvas-confetti) and **Apache-2.0** (class-variance-authority, TypeScript) — all of which permit use, modification, and distribution with attribution.